Running OPTI-GAS

This guide explains how to run the project in different scenarios.

1. Normal setup: Python already installed

Open a terminal in:

```
C:\Users\ANDREI\Documents\Opti-Gas
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it:

```
.venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install -r requirements.txt
```

Create your environment file if needed:

```
copy .env.example .env
```

Run the app:

```
python app.py
```

Open:

```
http://127.0.0.1:5000
```

2. If `python` does not work but Python is installed

Some Windows setups use `py` instead of `python`.

Create the virtual environment:

```
py -m venv .venv
```

Activate it:

```
.venv\Scripts\Activate.ps1
```

Install dependencies:

```
py -m pip install -r requirements.txt
```

Run the app:

```
py app.py
```

3. If Python is not installed at all

Install Python 3.12 or newer from the official Python website:

```
https://www.python.org/downloads/
```

Important during installation:

- enable `Add python.exe to PATH`
- install `pip`

After installation, close and reopen the terminal, then follow the normal setup.

4. If PowerShell blocks virtual environment activation

Run:

```
Set-ExecutionPolicy -Scope Process Bypass
.venv\Scripts\Activate.ps1
```

This only affects the current terminal session.

5. Run in VS Code

Open the `Opti-Gas` folder in VS Code.

Open the integrated terminal and run:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

If VS Code asks you to select a Python interpreter, choose:

```
.venv\Scripts\python.exe
```

6. Validate station data only

If you only want to check `stations.json`:

```
python scripts\validate_stations.py
```

Or:

```
py scripts\validate_stations.py
```

Expected success output:

```
OK: 76 stations loaded
OK: All required fields present
OK: All coordinates within Tagum City bounds
OK: Duplicate station names are allowed
```

7. If you only want to clean temporary files

Run:

```
powershell -ExecutionPolicy Bypass -File .\scripts\cleanup.ps1
```

This removes repo temp/cache clutter such as:

- `.tmp`
- `__pycache__`
- `pytest-cache-files-*`
- `tmp*`

8. If the app opens but the station cards load slowly

This is usually caused by one or more of these:

- browser geolocation delay
- slow internet for ORS or OSRM routing
- first-time route calculation for many stations

What to do:

- wait for location access to finish

- make sure internet is working
- refresh once after the first successful load
- keep the server running so route cache can help

9. If the page does not reflect code changes

Do this:

1. stop Flask with `Ctrl+C`
2. run `python app.py` again
3. hard refresh the browser with `Ctrl+F5`

10. If dependency install fails

Try upgrading pip first:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Or with `py`:

```
py -m pip install --upgrade pip
py -m pip install -r requirements.txt
```

11. If port `5000` is already in use

Temporary option:

```
$env:FLASK_PORT="5001"
python app.py
```

Then open:

```
http://127.0.0.1:5001
```

12. If you need the bundled runtime used in this workspace

In this workspace, a bundled Python runtime has been used successfully at:

```
C:\Users\ANDREI\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe
```

You can run the app with:

```
$env:PYTHONPATH='.vendor'
& 'C:\Users\ANDREI\.cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe' app.py
```

This is useful if local Python is missing or broken, but it is specific to this machine/workspace.

13. Recommended everyday commands

Run app:

```
python app.py
```

Validate stations:

```
python scripts\validate_stations.py
```

Clean temp files:

```
powershell -ExecutionPolicy Bypass -File .\scripts\cleanup.ps1
```